HPC OPTIMIZATION CentraleSupélec × CEA-DAM · Academic · Feb.–Apr. 2026

Optimal Routing under Blast-Exposure Constraints

Coupling high-fidelity blast simulation, Bayesian surrogate modeling and graph search on HPC infrastructure

Project team	Jenaya Ali Desmé, Tancrède Amiel, Merwan Bennani, Titouan Gay-Cortijo, Fleurine Jaegler, Joseph Leger
Partner supervision	Teddy Chantrait (technical supervision), Stéphane Jaouen (scientific lead) – CEA-DAM
Academic referent	John Cagnol – CentraleSupélec
Version	Public, October 2026

About this document

This is **not the original deliverable**. It is a reworked public version of the two documents the team submitted in April 2026: the final project report and the technical report of the 3D pipeline. They have been translated from French into English and merged into a single document, and every element covered by the confidentiality agreement between the partners has been removed:

- all source code and code excerpts;
- the internal design of the ARMEN simulation code;
- details of the computing infrastructure (paths, accounts, environments, hardware configuration);
- simulation outputs, except aggregated metrics and one illustrative damage map of the 2D test case (Figure 4).

Methods, equations, design rationale and results are otherwise preserved, and the rewording is editorial. The subject is an **academic exercise** designed by CEA-DAM to serve the pedagogical goals of the course and to give students access to its computing resources. It does not describe an operational application of the organization.

Executive summary

Problem. A device that may detonate must be moved from a point A to a point B across a built environment. Picture an explosive-ordnance-disposal team taking an unexploded device to a destruction site. The task is to find the route that minimizes the damage an accidental detonation would cause to the surrounding buildings, weighted by their criticality, while remaining practicable for the transport vehicle. The reference model is ARMEN, the high-performance hydrodynamics code of CEA-DAM. One simulation takes about 10 minutes in 2D and about 2 hours on 48 cores in 3D, so evaluating every candidate position of a 50×50 grid would require months of computation.

Approach. The team built a modular pipeline with three layers:

1. an analytical **Kingery–Bulmash surrogate**, augmented with building and terrain masking, provides a physics-based prior in seconds;
2. a **Gaussian process** learns only the residual between ARMEN and the surrogate, and an **active acquisition loop** chooses the few positions worth simulating;
3. a **graph search** computes the route on the corrected danger map: parallel Delta-Stepping in 2D, and a direction-aware A* that also penalizes turns and slope changes in 3D.

Damage is scored against the French ICPE overpressure thresholds and weighted by building criticality.

Results. In 2D, the loop converged after **19 simulations out of 10 000 candidate positions (0.2 %)**. Relative uncertainty along the route corridor fell from 14.5% to 4.0%, below the 5% stopping threshold. The 3D pipeline, which is the final deliverable, runs end to end on the partner's supercomputer and was demonstrated end to end at the final review with the partner. Six 3D simulations are integrated, and once the map is available a route is computed in under 10 seconds.

Scope and outlook. Under the time constraints of the course, several components were implemented as simpler proof-of-concept versions. They give valid results but are less general, less optimal or less scalable than the actual intended designs they approximate. Each section therefore closes with the optimal version of its concept. The main perspectives are neural-operator surrogates, geometry-aware kernels, decision-focused batch acquisition and a compiled, distributed path-search engine.

My contribution — Tancrède Amiel

- **Solution architecture and methodology**: problem scoping, decomposition into tractable subproblems, and selection of the methods that address them (multi-fidelity surrogate modeling, Bayesian optimization with active learning, graph-based path planning), designed as an end-to-end workflow before implementation (Section 3.2).
- **Problem formulation and objective function**: the cumulative damage plus angular criterion and its nondimensionalization (Section 3.1), and the damage-scoring design (Section 3.3).
- **HPC pipeline**: participation in its implementation on the partner's cluster.
- **Project orchestration**: a cross-functional coordination role and the team's strategic decisions on the overall approach.

Contents

Executive summary **1**

1 Introduction **3**
1.1 An academic exercise in partnership . . . 3
1.2 Motivating scenario: moving an unexploded device . . . 3
1.3 Objective and approach . . . 3

2 Problem analysis **5**
2.1 Formal statement . . . 5
2.2 The ARMEN simulator . . . 5
2.3 Modeling assumptions . . . 5
2.4 The computational challenge . . . 6

3 Solution design **7**
3.1 Objective function . . . 7
3.2 Architecture overview . . . 8
3.3 Damage scoring . . . 8
3.4 Physics-based surrogate: Kingery–Bulmash . . . 10
3.5 Gaussian-process correction . . . 11
3.6 Active acquisition and stopping . . . 12
3.7 Path-search algorithm . . . 13

4 Implementation **14**
4.1 The 2D pipeline: methodological proof of concept . . . 14
4.1.1 Surrogate, ground truth and GP . . . 14
4.1.2 Path-coupled acquisition . . . 14
4.1.3 Graph and Delta-Stepping . . . 14
4.1.4 Orchestration . . . 14
4.2 The 3D pipeline: final deliverable . . . 15
4.2.1 Scene, terrain and nodes . . . 15
4.2.2 3D surrogate with raycasting and relief . . . 15
4.2.3 GP update . . . 16
4.2.4 Simulation loop on the cluster . . . 16
4.2.5 Direction-aware A* in 3D . . . 17
4.3 Synthesis: 2D, 3D and target versions . . . 18
4.4 Parallel computing . . . 19

5 Validation and results **20**
5.1 Validation strategy . . . 20
5.2 Results in 2D . . . 20
5.3 Status and performance in 3D . . . 21
5.4 Scalability of parallel Delta-Stepping . . . 21

6 Conclusion and perspectives **24**
6.1 Contributions . . . 24
6.2 Limitations and perspectives . . . 24

1 Introduction

1.1 An academic exercise in partnership

The project took place within the seventh thematic sequence (ST7) of the CentraleSupélec engineering curriculum, whose theme was HPC optimization. It was carried out with CEA-DAM, a major French actor in high-performance computing (HPC) for national-security and environmental-monitoring missions. The partner gave the student teams access to a supercomputer and to **ARMEN**, its massively parallel hydrodynamics code. Students thus worked with professional-grade simulation and HPC tools.

The roles were split between the two partners:

- **CEA-DAM.** Teddy Chantrait presented the subject and provided technical supervision. Stéphane Jaouen owned the scientific question. Together they set the functional requirements, validated scientific choices and assessed the deliverables at formal milestones.
- **CentraleSupélec.** John Cagnol, the academic referent, supervised the student teams. The teams had to understand and formalize the problem, design and implement a solution, and defend their choices in two formal reviews.

The partner stated explicitly that the subject is a mathematical exercise designed for the course and for the use of its computing infrastructure, not an internal operational application.

1.2 Motivating scenario: moving an unexploded device

The problem is inspired by demining operations. A team must move a suspect device, such as an unexploded ordnance discovered on a worksite, from where it lies (A) to a safe destruction site (B) through a built-up area. The brief was deliberately left open-ended, to encourage original interpretations. The device could detonate at any point along the way, so the route matters. The team framed the problem as follows:

- it should keep the charge away from places where a blast would cause the most damage, giving more weight to critical buildings such as hospitals or schools than to storage annexes;
- it must remain practicable for the vehicle, with no abrupt turns and no brutal slope changes.

This scenario frames the exercise; no operational data were used. Damage levels are graded against the overpressure thresholds of the French regulation on classified installations for environmental protection (ICPE), the reference framework for accidental industrial explosions.

1.3 Objective and approach

The goal is to determine, for a given environment, the route between A and B that minimizes the overpressure impact on surrounding structures. This requires coupling two components:

- a **physical simulation**: for a candidate position of the charge, ARMEN computes the pressure field over the scene. It is the elementary and expensive brick of the problem;
- an **optimization loop** that orchestrates simulator calls to compare candidate positions and extract a route.

The study proceeded in two stages: first a planar 2D map, then a 3D environment with realistic topography and three-dimensional building effects.

Implementation strategy. The course imposed a tight schedule. The team therefore favored *proof-of-concept* implementations: simpler, less general, and not always optimal or scalable versions of the designs, chosen because they could be delivered and validated in time. They produce valid and satisfactory results. Each part of this report describes what was implemented, then

closes with a box describing the *optimal version* of the concept, not all of which could be integrated into the final pipeline.

2 Problem analysis

2.1 Formal statement

Let $A, B \in \mathbb{R}^d$ ($d = 2$ or 3) be the start and end points, and $\Omega \subset \mathbb{R}^d$ the spatial domain. The scene contains structures $\mathcal{S} = \{S_1, \dots, S_n\} \subset \Omega$, each with an importance weight $c_i > 0$. A route is a parametrized curve $\gamma : [0,1] \to \Omega$ with $\gamma(0) = A$, $\gamma(1) = B$ and $\gamma(t) \notin \bigcup_i S_i$. Finally, $p(y, x_s)$ denotes the pressure at $y \in \Omega$ induced by a detonation at x_s, as computed by ARMEN.

A first natural objective weights, for each structure, the worst pressure it receives along the trip:

$$J_0(\gamma) = \sum_{i=1}^{n} c_i \max_{t \in [0,1]} p(S_i, \gamma(t)), \qquad \gamma^\star = \arg\min_{\gamma \in \mathcal{A}} J_0(\gamma). \tag{1}$$

It served as a baseline only. The retained criterion is a cumulative, nondimensionalized one (Section 3.1).

The set $\mathcal{A}$ of admissible routes is infinite-dimensional and must be parametrized. Three families were considered:

- waypoints interpolated by splines or Bézier curves, giving a search space $\mathbb{R}^{K \times d}$;
- compact geometric parametrizations;
- a discretization of Ω into a weighted graph, which reduces the problem to a shortest path.

The team chose the graph: it handles obstacles naturally and gives access to exact, well-understood algorithms. The constraints are connectivity from A to B, obstacle avoidance, a minimal clearance from walls (the transport device has a volume), and possibly bounds on length or curvature.

2.2 The ARMEN simulator

For this project ARMEN can be treated as a black box. Given the scene geometry and the position x_s of the charge, it solves the compressible Euler equations

$$\partial_t \rho + \nabla \cdot (\rho \mathbf{u}) = 0, \quad \partial_t (\rho \mathbf{u}) + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u} + p\, \mathbf{I}) = 0, \quad \partial_t (\rho e) + \nabla \cdot ((\rho e + p)\, \mathbf{u}) = 0, \tag{2}$$

where ρ is the density, $\mathbf{u}$ the velocity, p the pressure and e the total specific energy. The equations are solved on an adaptively refined Cartesian mesh, and buildings and terrain are treated as rigid walls. The quantity used downstream is the field of **peak pressure** over time, from which the overpressure on each structure is derived. The code is parallel and scriptable from Python, which made it possible to wrap it in an optimization loop.

2.3 Modeling assumptions

H1 *Decoupled positions.* The blast is computed independently for each candidate position; cumulative effects between successive positions are neglected, since a single device can detonate at most once.

H2 *Discrete parametrization.* The route is represented by a finite set of nodes, and the optimization acts on them.

H3 *Rigid structures.* Obstacles are perfectly rigid and their deformation under load is ignored.

H4 *Stationary environment.* The geometry does not change during the simulation, and dynamic effects of the moving charge are neglected.

H5 *Point-like but bulky carrier.* The carrier is treated as a point for the physics, but its volume forbids routes that graze walls.

H6 *Rare, uniform hazard.* An accidental detonation is a rare event with a constant probability λ per unit length traveled, independent of the position along the route (Section 3.1).

2.4 The computational challenge

Each evaluation of the objective requires one or more complete ARMEN runs. A 2D run takes about 10 minutes and a 3D run about 2 hours on 48 cores. Simulating the 2 500 positions of a 50×50 grid in 3D would thus cost about 5 000 hours of wall-clock time. Three design questions follow:

1. Which **derivative-free, sample-efficient** optimization strategy suits an expensive and possibly non-smooth objective?
2. How can independent simulations be **submitted in parallel** on the cluster?
3. How should the strategy cope with **resource limits**: available partitions, core quotas and maximum job duration?

These questions lead to the surrogate-assisted, active-learning design of Section 3.

3 Solution design

3.1 Objective function

The retained criterion is a convex combination of two nonnegative, dimensionless terms:

$$J(P) = \alpha\,\widetilde{J}_{\mathrm{dmg}}(P) + \beta\,\widetilde{J}_{\mathrm{ang}}(P), \qquad \alpha,\beta > 0, \quad \alpha+\beta = 1, \tag{3}$$

where P is a path of grid cells (M_i).

- The **damage term** penalizes routes through areas where a detonation would be most harmful.
- The **angular term** penalizes abrupt trajectories that could destabilize the transported material. At a given speed, a sharp change of heading means a large lateral acceleration, $a_\perp \sim v^2\kappa$ with κ the curvature. The turning angle ϕ_n between three consecutive cells M_{n-1}, M_n, M_{n+1} is therefore a geometric proxy for lateral loads, not a full dynamic model.

The combination of the two terms is a multi-criteria scalarization. The damage term alone, however, has an exact probabilistic meaning, derived below.

Nondimensionalization. The damage term is normalized by a reference maximum

$$D_{\max} = d_{\max} \sum_c w_c N_c, \tag{4}$$

where $d_{\max}$ is the maximal damage per cell, w_c the weight of building category c and N_c its number of cells. Both terms are also normalized by the largest cell size $\Delta s_{\max}$, and the angular term by π^2 since $\phi_n \le \pi$. With P° the interior cells of the path,

$$J(P) = \alpha \sum_{i\in P} \frac{S(M_i)\,\Delta s_i}{D_{\max}\,\Delta s_{\max}} + \beta\,\Delta s_{\max} \sum_{n\in P^\circ} \frac{\phi_n^2}{\pi^2\,\Delta s_n}, \tag{5}$$

which reduces on a regular grid ($\Delta s_i \equiv \Delta s$) to

$$J(P) = \alpha \sum_{i=1}^{N} \frac{S(M_i)}{D_{\max}} + \beta \sum_{n=2}^{N-1} \frac{\phi_n^2}{\pi^2}. \tag{6}$$

Probabilistic interpretation of the damage term. Under assumption H6, the arc length s at which the device detonates follows an exponential law of rate λ. The expected damage of a trip along γ, of length L, is therefore

$$\mathbb{E}[D(\gamma)] = \int_0^L \lambda e^{-\lambda s}\, S(\gamma(s))\,\mathrm{d}s = \lambda \int_0^L S(\gamma(s))\,\mathrm{d}s + R, \qquad |R| \le \tfrac{1}{2}\,\lambda^2 L^2 \max S, \tag{7}$$

since $|e^{-\lambda s} - 1| \le \lambda s$. For a rare event ($\lambda L \ll 1$), minimizing the expected damage amounts to minimizing $\int S\,\mathrm{d}s$, whose discretization is exactly the damage term $\sum_i S(M_i)\,\Delta s_i$ of (5). The cumulative criterion is thus derived rather than postulated, and it already penalizes length: a longer route accumulates more exposure.

Rejected alternatives. Two other formulations were discarded:

- An *average* of damage along the path. Routes of very different lengths crossing the same zones obtained nearly equal scores. By (7), this discards the exposure duration, which is exactly what the expected damage accumulates.
- An explicit *length* term $\gamma L(P)$. By (7), the cumulative damage already penalizes length, so such a term would double-count exposure. It would only be justified by costs unrelated to the blast risk, such as transport time.

Weights. Damage should dominate, so $(\alpha, \beta) = (0.7, 0.3)$ is the reference choice. Lowering α in favor of β gives more importance to the mechanical stability of the transport. For simplicity, the 2D runs used $\alpha = 1$, $\beta = 0$. The 3D pipeline reintroduces the angular regularization directly inside the path-search cost (Section 4.2.5).

Toward the optimal version

The angle ϕ_n depends on three consecutive nodes, so the β-term is *not* additive over the edges of the spatial graph, and a plain shortest-path algorithm cannot optimize it. Augmenting each state with its incoming direction makes the term edge-additive. This is precisely the state space of the 3D directional A* (Section 4.2.5). The optimal version therefore optimizes the full criterion (5) with $\beta > 0$ exactly on the augmented graph, in both 2D and 3D, with a single, documented penalty shape: either quadratic in ϕ_n as in (5), or linear as implemented in 3D.

3.2 Architecture overview

The solution is a modular pipeline that goes from the geometric description of a scene to an optimal route (Figure 1). The user provides four inputs:

1. the scene: building footprints and their ICPE categories;
2. the endpoints A and B;
3. the charge W in kilograms of TNT equivalent;
4. an *acceptable time*, which sets the number N of ARMEN runs and hence the accuracy–speed trade-off. For instance, $N = 0$ (surrogate only) answers in under a minute, $N = 5$ in a few hours, and $N = 20$ in about a day in 3D.

Modules exchange data only through standardized intermediate files. Any stage can therefore be resumed without recomputing the previous ones, and any module can be replaced independently.

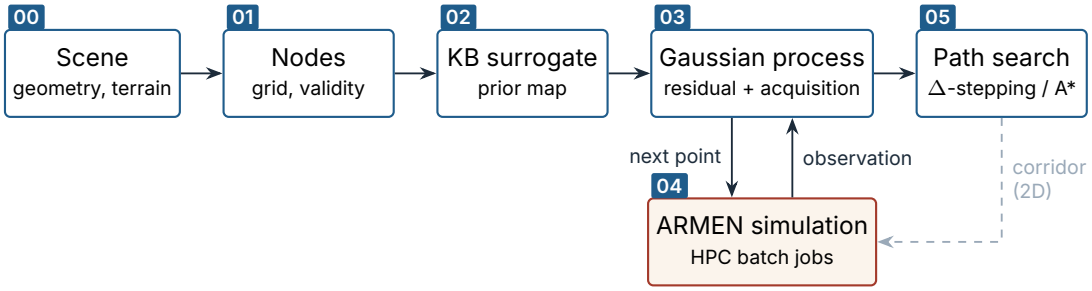


Figure 1: Pipeline architecture. The active-learning loop (03–04) calls ARMEN only where a simulation is most informative. In 2D the current optimal path also steers acquisition (dashed). The red module runs on the HPC cluster.

Two products share this architecture. The **2D pipeline** is the methodological proof of concept: it carries the more elaborate scoring, kernel and acquisition logic. The **3D pipeline** is the final deliverable to the partner: it carries the 3D physics, the automated HPC orchestration and the direction-aware path search. Their differences are summarized in Section 4.3.

3.3 Damage scoring

For each candidate charge position x_s, the pipeline needs a continuous, additive scalar score of the damage inflicted on the surrounding structures. After an ARMEN run, each mesh cell is assigned to the nearest building through a k-d tree, which gives the building's identifier and ICPE category. The score is then built in two steps: a *local damage function* g converts a cell's peak overpressure into an equivalent damage level, and a *criticality coefficient* d_k weights each

building k by its functional importance:

$$S(x_s) = \sum_k d_k \sum_{m \in \mathcal{M}_k} g(p_{\max}(m, x_s)). \tag{8}$$

Local damage function. A first model used a global logarithmic law,

$$s(p) = \frac{\ln(\mathrm{clip}(p, 20, 300)/20)}{\ln(300/20)} \in [0, 1] \qquad (p \text{ in hPa}). \tag{9}$$

It compresses high pressures and blurs the distinction between severe regimes, whereas damage tables show qualitative transitions rather than a continuous trend [1, 2].

The retained model interpolates smoothly between the five ICPE thresholds $(p_1, \dots, p_5) = (20, 50, 140, 200, 300)$ hPa, which correspond to window breakage, light damage, serious damage, domino effects and very serious damage. These thresholds are associated with the damage levels $G = (0,\ 0.03,\ 0.18,\ 0.70,\ 2.20)$. On each interval $[p_i, p_{i+1}]$, a cubic Hermite step $h(u) = 3u^2 - 2u^3$ is applied in log scale:

$$g(p) = G_i + (G_{i+1} - G_i)\, h\left(\frac{\ln p - \ln p_i}{\ln p_{i+1} - \ln p_i}\right), \qquad g = 0 \text{ below } p_1, \quad g = G_5 \text{ above } p_5. \tag{10}$$

Since $h'(0) = h'(1) = 0$, g is C^1 and its slope vanishes at every threshold. The choice of h is purely numerical. The strongly nonlinear hierarchy of the G_i, by contrast, carries the physics: damage grows slowly in superficial regimes and sharply in severe ones, where effects spread beyond the impacted cell [3]. This is consistent with blast fragility curves, which organize damage into discrete states rather than an affine law [4]. A known limitation is that g relies on peak overpressure alone and ignores impulse [2]; the explicit anchoring on ICPE thresholds partly compensates for it.

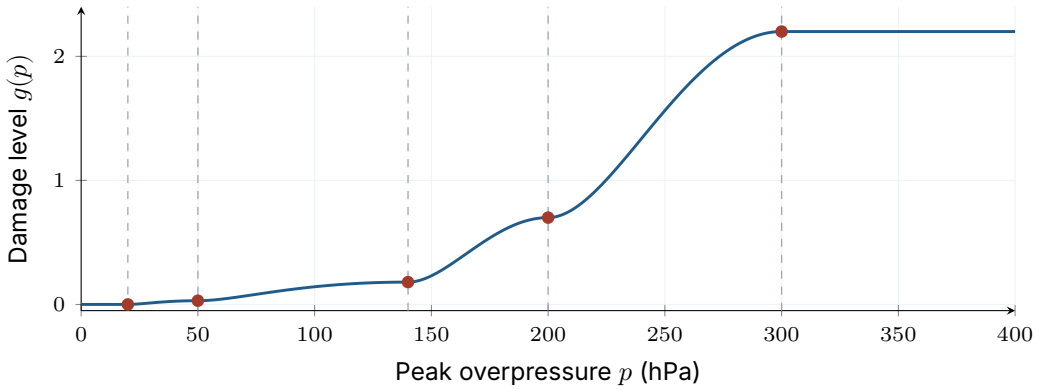


Figure 2: Local damage function $g(p)$ of (10), with the five ICPE thresholds (dashed) and their damage levels (dots).

Criticality weighting. The coefficients d_k express that damage to a strategic building cannot weigh the same as damage to a replaceable annex (Table 1). They were chosen by reading the community-resilience literature:

- NIST identifies critical facilities (hospitals, schools, command centers) as priorities because their loss slows collective recovery [5, 6];
- FEMA and the OECD stress the cascading effects of losing vital buildings [9, 10].

The value $d_2 = 2.5$ reflects the human weight of ordinary housing; IDMC counts 45.8 million disaster-related internal displacements in 2024 [7]. The jump to $d_3 = 7$ reflects the disproportionate social disruption caused by losing collective facilities; UNICEF reports 242 million students

affected by hazard-related school disruptions in 2024 [8]. Finally, $d_4 = 20$ adds a systemic premium for buildings whose loss has irrecoverable short-term cascading effects. These values are a reasoned modeling choice informed by the literature, not a calibration.

Cat.	Type	Examples	d_k
1	Secondary	Annexes, warehouses, replaceable buildings	1.0
2	Residential / economic	Housing, shops, offices	2.5
3	Collective facilities	Schools, cultural venues, public buildings	7.0
4	Strategic and vital	Hospitals, police, fire stations, command centers	20.0

Table 1: Criticality coefficients by ICPE category (2D pipeline).

What was implemented. The 2D pipeline uses g and the coefficients d_k, summed over cells as in (8). The 3D pipeline, delivered as a proof of concept, keeps the first-generation logarithmic score (9) with category weights $\omega = (1, 2, 3, 5)$. It aggregates each building by the 90th percentile of its cell scores rather than a sum (Section 4.2.2).

Toward the optimal version

- A **single shared scoring module** implementing g and d_k for both pipelines and for both the surrogate and ARMEN outputs.
- An explicit **aggregation choice**. A sum over cells depends on building size and mesh resolution; a percentile is size- and resolution-invariant but discards extent. A size-normalized sum, $|\mathcal{M}_k|^{-1} \sum_m g$ times a footprint factor, would separate the two effects cleanly.
- The addition of **impulse** through pressure–impulse damage curves, so that two blasts with the same peak but very different durations are no longer scored alike.

3.4 Physics-based surrogate: Kingery–Bulmash

Mapping the score over the whole domain without one ARMEN run per cell requires a fast empirical model. The Kingery–Bulmash (KB) curves [13] give the incident free-field peak overpressure P_{KB} as a function of the scaled distance

$$Z = \frac{R}{W^{1/3}} \quad [\mathrm{m\,kg^{-1/3}}], \tag{11}$$

where R is the source–target distance and W the charge in kg TNT equivalent. The curves are the reference of blast-resistant design [14]. Their usual closed-form implementation [15] is a polynomial fit in a transformed log-distance, $U = K_0 + K_1 \ln Z$ and $\log_{10} P_{\mathrm{KB}} = \sum_i C_i U^i$, with coefficient sets specific to successive ranges of Z.

What was implemented, and a limitation. The pipeline used its own single empirical polynomial fit in $\log Z$ over $Z \in [0.2, 40]$, whose coefficients are not reproduced here. A post-project check showed that this fit reproduces the reference curve only over a limited range of scaled distances and departs strongly from it elsewhere. Absolute surrogate pressures are therefore unreliable outside that range. The design partly absorbs this limitation:

- the damage score saturates above 300 hPa;
- the GP corrects the surrogate wherever ARMEN observations exist;
- in the 3D pipeline, the empirical drag factor (Section 4.2.2) may in practice have compensated for the overestimation.

Free-field pressure ignores obstacles, so each pipeline adds masking corrections:

- **2D.** A 2D raycast counts the walls between source and target and applies a shadow factor between 0.7 and 0.9 depending on distance. A residual pressure term P_0, calibrated on reference ARMEN runs, accounts for the confinement specific to the scene.
- **3D.** Exponential masking by buildings and by terrain is applied along 3D rays, followed by spatial smoothing (Section 4.2.2).

Evaluated in milliseconds per source in 2D, the surrogate serves as the *physical prior* of the Gaussian process.

Toward the optimal version

KB is a free-field law: it misses Mach reflection, channeling in street canyons and multiple reflections. Where these effects dominate, residuals are large and the GP needs many more simulations to correct the prior. The optimal surrogate would include reflected-pressure corrections, which are standard in blast engineering. In the longer term, it would be a learned operator trained on ARMEN runs (Section 6). Its masking parameters would be calibrated against ARMEN rather than set by hand, and its free-field law would be the standard range-wise KB formulation [15], validated against reference values.

3.5 Gaussian-process correction

The cost of ARMEN rules out direct evaluation: 2 500 runs on a 50×50 grid would take months. A Gaussian process (GP) [11] is used as a metamodel instead. Trained on a handful of real simulations, it interpolates the damage field over the whole domain in milliseconds. It also quantifies its own uncertainty point by point, which is precisely what an active acquisition strategy needs.

Residual learning. Rather than learning the full ARMEN score, whose variations are amplified by the building geometry, the GP learns the residual with respect to the KB prior [12]:

$$r(x) = f_{\text{ARMEN}}(x) - f_{\text{KB}}(x), \qquad f \sim \mathcal{GP}(f_{\text{KB}}, k). \tag{12}$$

The residual only captures the gap between simplified free-field physics and the real effects of confinement and diffraction. It is a much smoother function, so the GP converges in a few simulations where naïve learning would need an order of magnitude more.

Posterior. Let $\mathcal{D}_t = \{(x_i, r_i)\}_{i \le t}$ be the observations, $K = k(X, X) + \sigma_n^2 I$ their Gram matrix including an optional nugget σ_n^2, and $\mathbf{k}(x) = k(x, X)$. The posterior of the residual and the resulting map are

$$\bar{r}(x) = \mathbf{k}(x)^\top K^{-1} \mathbf{r}, \qquad \sigma_r^2(x) = k(x, x) - \mathbf{k}(x)^\top K^{-1} \mathbf{k}(x), \qquad \mu_{\text{post}}(x) = f_{\text{KB}}(x) + \bar{r}(x). \tag{13}$$

Hyperparameters are fitted by maximizing the marginal likelihood.

Kernel. The design choice is the Matérn 5/2 kernel, with $\Delta = x - x'$:

$$k(x, x') = C\Big(1 + \tfrac{\sqrt{5}\,\|\Delta\|}{\ell} + \tfrac{5\|\Delta\|^2}{3\ell^2}\Big) \exp\Big(-\tfrac{\sqrt{5}\,\|\Delta\|}{\ell}\Big), \tag{14}$$

with an initial correlation length $\ell \approx 20\,\text{m}$. Its sample paths are twice differentiable, one order more than Matérn 3/2. That regularity suits free-field pressure, whose gradients vary smoothly away from structures. Unlike the infinitely differentiable RBF kernel, it does not over-smooth the sharp transitions created by building edges [16]. A small white-noise nugget stabilizes the inversion of K and absorbs tiny numerical errors from the simulator.

What was implemented. The 2D pipeline uses Matérn 5/2 with a nugget. The 3D pipeline uses an RBF kernel without nugget, whose length scale is fitted only once three observations are available (Section 4.2.3).

Main limitation: Euclidean distance. The kernel judges two points on either side of a wall to be close, although blast propagation between them is strongly attenuated. A geodesic distance that goes around obstacles, computable for instance by the Fast Marching Method [19], would be physically more faithful. Plugging non-Euclidean distances into standard kernels, however, does not in general preserve the positive semi-definiteness required for a valid covariance [11].

Toward the optimal version

- Matérn 5/2 with a nugget in *both* pipelines.
- Geometry injected in a way that keeps the covariance valid: a stationary kernel applied to *physically informed features*, such as KB shadowing factors or line-of-sight indicators, rather than raw coordinates.
- Non-stationary kernels or deep-kernel learning, to capture the discontinuities created by masking.

3.6 Active acquisition and stopping

Choosing where to run the next simulation is the heart of Bayesian optimization [17]. The classical criterion is the Upper Confidence Bound (UCB) of Srinivas et al. [18], written here for a search of the *maxima* of f:

$$x_{t+1} = \arg\max_x \left(\mu_t(x) + \sqrt{\beta_t}\,\sigma_t(x)\right), \qquad \beta_t = 2\ln\!\left(\frac{N_v\, t^2\pi^2}{6\delta}\right), \tag{15}$$

where N_v is the number of valid nodes and $\delta = 0.1$ the confidence level. The search for minima uses the mirrored bound $\arg\max\left(-\mu_t + \sqrt{\beta_t}\,\sigma_t\right)$. A large β_t favors exploration of uncertain zones, a small one exploitation.

Routing, however, needs neither the maximum nor the minimum of f: it needs an *accurate map along the optimal route*. The two pipelines answer this differently:

- **2D: path-coupled acquisition.** Acquisition targets the uncertainty inside a corridor around the current optimal path, computed by Delta-Stepping on the GP mean, while keeping some global exploration (Section 4.1.2). The loop stops when the relative cumulative uncertainty on the corridor falls below 5%:

$$\sigma_{\text{cum,rel}} = \frac{\sum_{c\in\mathcal{C}} \sigma(c)}{\sum_{c\in\mathcal{C}} \mu(c)} < 0.05. \tag{16}$$

  This criterion is consistent with a sum-of-costs objective: what makes the solution reliable is the uncertainty accumulated along the route, not the local uncertainty peak. Path stability between iterations, measured by the Hausdorff distance, provides a second check.

- **3D: standard UCB on danger** (15), which drives simulations toward the most dangerous nodes and reduces uncertainty where the prior predicts the strongest effects (Section 4.2.3).

Toward the optimal version

- A **decision-focused** acquisition: simulate where the information is most likely to change the route, for example by maximizing the expected reduction of uncertainty on the path cost, or the probability that the optimal path changes.
- A **batch** acquisition of q points per round (for example with local penalization), to exploit the cluster's capacity to run several simulations concurrently.

- A single stopping rule combining (16) with path stability, shared by both pipelines.

3.7 Path-search algorithm

The domain is discretized on a regular grid in which each cell becomes a node. Cells occupied by buildings get an infinite score and are therefore impassable. Each node is connected to its 8 neighbors in 2D and to its 26 neighbors in free 3D space. The weight of an edge is

$$w(u \to v) = s(v)\, d(u,v), \qquad d(u,v) \in \{1, \sqrt{2}, \sqrt{3}\}, \tag{17}$$

where $s(v)$ is the danger score of the destination node and $d(u,v)$ the geometric step factor (axis, 2D diagonal, 3D diagonal), in units of the grid spacing. A shortest path thus minimizes danger accumulated along the actual distance traveled.

Three classical algorithms were compared:

- **Dijkstra** [20], in $O((|V| + |E|) \log |V|)$, is inherently sequential: each step depends on the global minimum of the queue.
- **A*** [21] adds an admissible heuristic that guides exploration and reduces the number of expanded nodes. Its main loop is still sequential.
- **Delta-Stepping** [22] groups tentative distances into buckets of width Δ. The smallest non-empty bucket is processed in a *light* phase, which relaxes edges of weight at most Δ and may re-insert nodes into the current bucket, then a *heavy* phase for edges of weight above Δ, which cannot re-enter it and thus guarantees termination. Nodes within a bucket can be relaxed independently, which maps naturally onto MPI processes.

Delta-Stepping was retained for the 2D pipeline and for the volumetric 3D graph. A $100 \times 100 \times 30$ grid with 26-connectivity already has 300 000 nodes and about 8 million edges, which makes it the target for parallelism. Δ is set automatically to the 25th percentile of the finite edge weights, an empirical balance between the number of buckets and intra-bucket parallelism.

For the final 3D product, the vehicle moves on the terrain surface, a 2.5D graph of 2 500 nodes. A sequential, **direction-aware A*** is then both sufficient and more expressive, since it can penalize turns and slope changes (Section 4.2.5).

Toward the optimal version

A single engine running Delta-Stepping or A* on the *direction-augmented* state graph, so that the full objective (5) is optimized exactly. The engine would be compiled and distributed (Section 5.4). Parallel Delta-Stepping is worth its synchronization cost only for volumetric or very large graphs; on surface graphs of a few thousand nodes, sequential A* is faster.

4 Implementation

Both pipelines are sets of independent modules that communicate only through intermediate files. This section describes how each concept of Section 3 was implemented, including the non-obvious choices that make the code work.

4.1 The 2D pipeline: methodological proof of concept

4.1.1 Surrogate, ground truth and GP

- **Surrogate score.** For a candidate source, the KB model (with 2D raycasting) gives the overpressure at each building probe, which is converted into damage by g and d_k. A KB map is precomputed on the 100×100 evaluation grid. It initializes the map and provides the prior value at training points.
- **Ground truth from ARMEN.** The simulator stores *absolute* pressure, so peak values are converted to overpressure, $p_{\mathrm{hPa}} = (p_{\mathrm{Pa}} - 10^5)/100$. Each building probe is then matched to its nearest mesh cell with a k-d tree, and the same scoring chain as the surrogate is applied. The difference between both scores is the GP training residual.
- **GP.** The kernel is $C \times \mathrm{Matérn}(\nu = 5/2) + \text{white noise}$. The initial length scale is 20 m with bounds $[5, 100]$ m, the nugget starts at 10^{-3} with bounds $[10^{-6}, 1]$, and the marginal likelihood is optimized with 10 random restarts to avoid local optima. Targets are standardized before fitting for numerical conditioning.
- **Damage map.** The final map is the KB map plus the predicted residual over all 10 000 grid points, together with the pointwise standard deviation.
- **Initial design.** Without any simulation the map degenerates to pure KB with zero uncertainty, which gives acquisition nothing to work with. The loop is therefore seeded with a Latin-hypercube design of 15 points.

4.1.2 Path-coupled acquisition

The corridor $\mathcal{C}$ is the set of grid points within 8 m of the current Delta-Stepping path computed on μ. The acquisition score combines exploitation of corridor uncertainty and exploration of safe but unexplored areas:

$$a(x) = \beta \, \frac{\sigma(x)}{\max \sigma} \, \mathbf{1}_{\mathcal{C}}(x) + (1 - \beta) \, \frac{\sigma(x)/(\mu(x) + 10^{-3})}{\max_y \sigma(y)/(\mu(y) + 10^{-3})}, \qquad \beta = 0.7. \tag{18}$$

Building cells and points closer than 3 m to an existing simulation are masked out before taking the maximum. The module also exposes UCB, expected improvement and pure-uncertainty strategies; the reported runs use the path-coupled rule with the stopping criterion (16).

4.1.3 Graph and Delta-Stepping

The graph described in Section 3.7 is precomputed from the score map. Endpoints are snapped to the nearest graph node with a k-d tree. The resulting path is exported for visualization: a heatmap with a colored polyline in 2D, and terrain, building volumes and the path polyline in 3D. Three metrics are extracted: total cumulative danger, number of nodes traversed and mean danger per node.

4.1.4 Orchestration

ARMEN runs are batch jobs that execute asynchronously, decoupled from the main script. The 2D pipeline is therefore driven by a central orchestrator with two modes (Figure 3):

- **Initialization.** A coarse path is computed on KB scores over a 5 m grid. The grid is then adaptively refined to a 2–5 m step around the resulting corridor, with automatic tightening

where the score gradient is strong. Finally, initial points are selected, and for each of them a self-contained simulation case is generated by programmatically patching a template input file. The script then exits, leaving job submission to the user.

- **Resume.** Finished simulations are integrated into the GP, the damage map is re-interpolated, and the fine path is recomputed on GP scores. Convergence is then checked on two criteria: mean GP uncertainty below a threshold, and Hausdorff distance between successive paths below a tolerance. If the loop has not converged, the next acquisition point is selected, its case is generated and the script pauses again.

Modules never call each other directly. Every exchange goes through intermediate files on the cluster's shared file system, which makes each phase restartable.

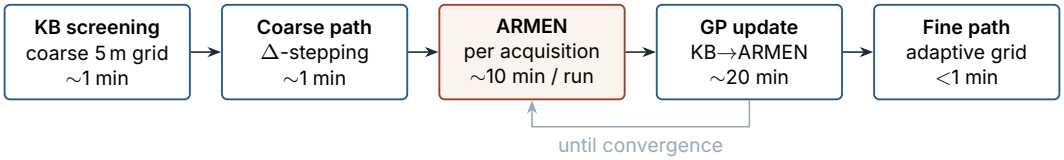


Figure 3: Execution flow of the 2D pipeline, with typical durations: fast KB screening and refinement, then an active loop of ARMEN runs and GP updates.

Toward the optimal version — 2D loop

A fully automatic loop in which each finished simulation triggers the next one through job dependencies (as in 3D, Section 4.2.4), combined with batch acquisition and the unified stopping rule of Section 3.6.

4.2 The 3D pipeline: final deliverable

The 3D pipeline chains six modules (Figure 1): scene parsing, node placement, KB surrogate, GP update, ARMEN simulation with observation update, and path search. It is operational end to end on the partner's cluster.

4.2.1 Scene, terrain and nodes

The scene provides building footprints with heights and ICPE categories, and a terrain point cloud of about 10 700 points. Ground elevation is interpolated by inverse-distance weighting over the four nearest terrain points. Candidate nodes form a regular 50×50 grid (2 500 nodes) over the 100×100 m scene, lifted 1 m above the ground (the carrier height). Nodes whose projection falls inside a building are invalid.

4.2.2 3D surrogate with raycasting and relief

Probes and pressure. Each building is sampled by *probes* placed at its polygon vertices and centroid, at ground level plus 2 m. For a source at node x, the free-field KB pressure is computed at each probe, with the distance clipped at $R \geq 1$ m. It is then corrected by three factors:

- *Building masking.* If the source–probe segment crosses a building outline, an attenuation factor is applied for each occluding building k:

$$f_{\text{bat}} = \prod_{k \in \mathcal{O}} \left[\alpha_{\min} + (1 - \alpha_{\min})\, e^{-d_k/d_0} \right], \qquad \alpha_{\min} = 0.02,\ d_0 = 5 \text{ m}, \tag{19}$$

  where d_k is the distance from obstacle k to the probe. A probe right behind a wall keeps almost full pressure, and the shadow deepens with distance behind the obstacle.

- *Terrain masking.* The 3D ray is sampled at 20 points between 10% and 90% of its length. If the terrain rises above the ray, the same law is applied with d_{shock}, the distance from the *last* occlusion point (the crest nearest the target) to the probe.

- *Drag.* An extra empirical decay $f_{\text{drag}} = e^{-R/10\,\text{m}}$ is applied.

The effective overpressure is $P_{\text{eff}} = 10\,P_{\text{KB}}\,f_{\text{bat}}\,f_{\text{terr}}\,f_{\text{drag}}$ in hPa.

Diffraction smoothing. Each probe's pressure is replaced by the mean over all probes within 5 m, a crude model of diffraction.

Score. Each cell gets the logarithmic score (9) weighted by its category, $s_{\text{cell}} = s(P)\,\omega(c)$ with $\omega = (1, 2, 3, 5)$, after assignment to the nearest probe. Each building is summarized by the 90th percentile of its cell scores, $S_b = \text{P}_{90}\{s_{\text{cell}}\}$, which is more representative than the mean because shadowed façades dilute it less. The scene score is $\sum_b S_b$.

Parallelism. Nodes are independent. A process pool sized from the scheduler's core allocation distributes them, collects results as they complete regardless of order, and shares read-only data copy-on-write. The full 50×50 map takes about 2 minutes.

Implementation details worth knowing. Two terrain models coexist: nodes use the IDW interpolation, whereas the terrain-occlusion test uses nearest-neighbor interpolation. The drag factor also stacks a strong exponential decay on top of a law that already decays with distance. At 50 m it divides pressure by about 150.

Toward the optimal version — 3D surrogate

- A single terrain model shared by all modules.
- The standard KB formulation in place of the empirical fit (Section 3.4), and removal of the drag factor. Candidly, this factor may have been masking the fit's overestimation of far-field pressure rather than modeling a physical effect.
- Calibration of $(\alpha_{\min}, d_0)$ against ARMEN, by fitting them to the residuals.
- The second-generation scoring $(g,\, d_k)$ of Section 3.3.

4.2.3 GP update

Normalization. KB scores are standardized over the valid nodes, $\tilde{s} = (S_{\text{KB}} - \hat{\mu})/\hat{\sigma}$. Observations are standardized with the same constants, and the GP is trained on residuals in normalized units, $r_i = \tilde{y}_i^{\text{ARMEN}} - \tilde{S}_{\text{KB}}(x_i)$. The posterior mean is then denormalized: $\mu_{\text{post}} = (\tilde{S}_{\text{KB}} + \bar{r})\,\hat{\sigma} + \hat{\mu}$.

Kernel and fitting. The kernel is $\sigma_f^2 \exp(-\|x - x'\|^2/2\ell^2)$ on the 3D coordinates, with $\ell_0 = 20$ m and bounds $[1, 100]$ m. The length scale is optimized by L-BFGS-B with 3 restarts only once $t \geq 3$; with fewer points, hyperparameters stay fixed.

Cold start. With no observation, the map is the KB prior and the uncertainty is uniform, $\sigma \equiv \hat{\sigma}$. UCB then reduces to the arg-max of the KB score, so the first simulation is placed at the node the prior deems most dangerous.

Acquisition. UCB (15) uses $t = n_{\text{obs}} + 1$ and a floor $\beta_t \geq 0.1$. Because sources are always grid nodes, the KB value associated with an observation is exact: it is that of the nearest valid node.

4.2.4 Simulation loop on the cluster

An orchestration script reads the next point proposed by the GP and creates a per-iteration directory. It instantiates the simulation, post-processing and update job scripts from templates

by substituting the source coordinates. It then submits a chain of three jobs, each starting only after the successful completion of the previous one:

1. the **ARMEN simulation**, about 2 hours on 48 cores;
2. the **post-processing**, which extracts peak pressures in about 8 minutes;
3. the **Bayesian update**: the ARMEN score is computed (overpressure conversion, nearest-probe assignment, per-building 90th percentile), the triplet $(x_t, S_{\text{ARMEN}}, S_{\text{KB}})$ is appended to the observation file, the GP is refitted and the next point is proposed.

One command thus advances the loop by one iteration, and each completed chain leaves the state ready for the next. At delivery, six complete 3D simulations had been integrated.

4.2.5 Direction-aware A* in 3D

State space. The path is sought on the 8-connected surface grid of valid nodes. Each state is a pair (node, incoming direction), with direction $d \in \{1, \dots, 8\}$ or *none* at the start. This gives $N(N_d + 1) = 22\,500$ states for $N = 2\,500$ nodes and $N_d = 8$. The augmentation is what makes turn and slope penalties edge-additive.

Penalties. With $\hat{u}_{\text{in}}, \hat{u}_{\text{out}}$ the unit 3D vectors of the incoming and outgoing edges, the horizontal and vertical penalties are

$$p_h = \frac{1}{\pi} \arccos\big(\text{clip}(\hat{u}^{2D}_{\text{in}} \cdot \hat{u}^{2D}_{\text{out}}, -1, 1)\big) \in [0, 1], \qquad p_v = \min\Big(\frac{|\varphi_{\text{out}} - \varphi_{\text{in}}|}{\pi/2},\, 1\Big), \quad \varphi = \arctan \frac{u_z}{\|u_{xy}\|}. \tag{20}$$

Cost. The cost of moving from state (v, d_{in}) to (v', d_{out}) is

$$C = w(v')\, \|v' - v\|_3 + \alpha_h\, p_h + \alpha_v\, p_v, \qquad w(v) = \mu_{\text{post}}(v)^{\eta}, \tag{21}$$

with $\eta = 3$ and $\alpha_h = \alpha_v = 50$.

- The exponent $\eta > 1$ forces large detours around dangerous zones. As $\eta \to \infty$, minimizing $\sum \mu^{\eta}\, \ell$ approaches minimizing the worst danger encountered, so η interpolates between the cumulative criterion ($\eta = 1$) and the minimax criterion of (1).
- The penalty weights are calibrated to be commensurate with μ^{η}, which can far exceed 1 on dangerous nodes. The non-exponentiated variant uses $\alpha_h = 0.3$, $\alpha_v = 0.2$.

Positivity. The GP posterior can be slightly negative, and negative weights break the correctness of Dijkstra and A*. When $\min \mu_{\text{post}} < 0$, the map is therefore shifted to $\mu \leftarrow \mu - \min \mu + 10^{-3}$ before exponentiation.

Heuristic. The heuristic is $h(v) = w_{\min}\, \|v_{xy} - B_{xy}\|_2$, with B the requested end point and $w_{\min}$ the smallest weight over valid nodes. The search starts and ends at the valid nodes nearest to A and B, denoted v_A and v_B.

Proposition — the heuristic is consistent, and A* returns an optimal path

For any transition $u \to v$, $C(u, v) \geq w(v)\|v - u\|_3 \geq w_{\min}\|v_{xy} - u_{xy}\|_2 \geq h(u) - h(v)$. The first inequality holds because the penalties are nonnegative, the second because the 3D length dominates its horizontal projection, and the third by the triangle inequality. Hence h is consistent.
Because the search ends at the snapped node v_B rather than at B itself, $h(v_B) = w_{\min}\|v_{B,xy} - B_{xy}\|_2 = c \geq 0$. This is a small constant, shared by all goal states (one per incoming direction). The shifted heuristic $h - c$ is consistent and vanishes at the goal, hence admissible. It orders the priority queue exactly as h does. A* therefore returns an optimal path on the augmented graph.

Variants. Six variants were implemented to isolate the effect of each ingredient (Table 2). The full variant finds the global optimum in under 10 seconds on the 50×50 grid. It reports the path, its mean (non-exponentiated) danger, its 3D length, the cumulative elevation change and the angles along the path.

Variant	Algorithm	Risk exponent η	Turn penalty	Slope penalty
Baseline	Dijkstra	–	–	–
Directional 2D	Dijkstra	–	✓	–
Directional 3D	Dijkstra	–	✓	✓
Risk-averse	A*	✓	–	–
Risk-averse directional 2D	A*	✓	✓	–
Risk-averse directional 3D	**A***	✓	✓	✓

Table 2: Path-search variants of the 3D pipeline. The last one is the product version.

Toward the optimal version — 3D path search

By consistency, the first goal state popped from the queue is already optimal. The implementation keeps expanding to compare all incoming-direction goal states, so stopping at the first one is a free speed-up. Practicability constraints (maximum slope, forbidden zones) can be added as hard constraints by pruning edges. The same engine would also run the 2D objective (5) with $\beta > 0$.

4.3 Synthesis: 2D, 3D and target versions

The two pipelines were developed under time pressure and do not make the same choices (Table 3). In each case the delivered choice gives valid results; the right column gives the version each concept points to.

Component	2D pipeline (proof of concept)	3D pipeline (deliverable)	Target version
Damage score	ICPE spline $g(p)$, $d_k = (1, 2.5, 7, 20)$, sum over cells	Logarithmic score, $\omega = (1, 2, 3, 5)$, P90 per building	g, d_k, size-normalized aggregation, impulse
Surrogate	KB + wall counting (0.7–0.9) + calibrated P_0	KB + exponential building and terrain masking, drag, smoothing	Reflection-aware, ARMEN-calibrated, then learned operator
GP kernel	Matérn 5/2 + nugget	RBF, no nugget, MLE from $t \geq 3$	Matérn 5/2 + nugget, physics-informed features
Acquisition	Path-coupled corridor rule ($\beta = 0.7$)	UCB on danger	Decision-focused, batch
Stopping	$\sigma_{\text{cum,rel}} < 5\%$ + path stability	Simulation budget	Unified rule
Path search	Delta-Stepping (MPI), $\alpha = 1, \beta = 0$	Directional A*, μ^3, turn and slope penalties	One engine on the augmented graph, full J
Orchestration	Orchestrator with init/resume modes	Chained batch jobs, one command per iteration	Fully automatic loop

Table 3: Implemented choices in each pipeline and the target version of each concept.

4.4 Parallel computing

Most of the computational cost lies inside ARMEN, which is parallel internally. The project's own parallel code consists of two parts:

- the embarrassingly parallel evaluation of the 3D surrogate (Section 4.2.2);
- the MPI implementation of Delta-Stepping.

Graph decomposition. Delta-Stepping relies on a spatial decomposition of the graph. In *free 3D* mode, nodes are split into slabs of vertical layers along z. In *vehicle* mode, the valid surface nodes (terrain plus carrier height) are split evenly across processes.

One iteration. At each iteration:

1. relaxations internal to a process are applied locally;
2. relaxations crossing a partition boundary are exchanged with an all-to-all;
3. global distances are synchronized by an all-reduce with the `MIN` operator;
4. another all-reduce determines the minimal active bucket.

An early-stopping test ends the run as soon as the active bucket exceeds the bucket of the current distance to the target.

Four processes suffice for the $100 \times 100 \times 30$ grid. All intermediate results (graph, damage map, path) are saved so that any stage can be resumed after an interruption, and paths are exported in a portable text format for local visualization.

5 Validation and results

5.1 Validation strategy

Validation aims at two things. The route returned through the GP must be mathematically converged, and it must be faithful to what an exhaustive (and unaffordable) ARMEN exploration would have produced. Three axes structure it:

1. **Physical prior.** KB predictions are compared with ARMEN on isolated test points. The aim is not zero error, since the GP exists to learn the residual, but correct orders of magnitude for free-field overpressure and for primary shadowing.
2. **Convergence of active learning.** The relative cumulative uncertainty $\sigma_{\text{cum,rel}}$ on the corridor should decrease (quasi-)monotonically across iterations, which shows that acquired points are informative. The algorithm declares convergence by itself when it falls below 5%.
3. **Stability of the decision.** The Hausdorff distance between successive paths should vanish: once new simulations no longer change the route macroscopically, the decision is stable despite the interpolation.

Scope of the validation. The results below quantify the second axis, *convergence*. They do not quantify accuracy against ARMEN, and the first and third axes are not reported numerically. This follows from the brief:

- no independent test data set was provided;
- the goal of the project was a functional tool that represents the physics in a believable way, within a three-month course.

In a professional setting, systematic testing would be part of the process. It would require held-out ARMEN runs to measure the error of the damage map, for instance by leave-one-out cross-validation, and a comparison of the returned route with a dense reference on a small scene. That would make for a substantially larger project.

5.2 Results in 2D

Setup. The test scene is a $100 \times 100\,\text{m}$ urban map with 15 buildings of different criticality classes. The task is to connect $A(5, 5)$ to $B(95, 95)$ while minimizing cumulative exposure. The loop starts from a 15-point Latin-hypercube design, with $\beta = 0.7$, a slight bias toward exploitation of the corridor.

Convergence. Table 4 and Figure 5 trace the loop. Iterations alternate between corridor points and exploratory points. The exploratory point of iteration 2 lies far from the route; its run served to confirm or rule out bypass routes.

Iteration	Simulations	$\sigma_{\text{cum,rel}}$	Role of the acquired point
0	15	14.46%	Initial Latin-hypercube design
1	16	9.89%	Inside the uncertain corridor
2	17	6.03%	Exploratory, far from the route: checks bypass options
3	18	5.29%	Back in the corridor
4	19	**4.03%**	Below the 5% threshold: loop stops

Table 4: Active-learning trajectory on the 2D case.

Route. The final Delta-Stepping route has 106 steps and successfully bypasses the most critical buildings, whose surroundings carry prohibitive scores. A minimax criterion would minimize

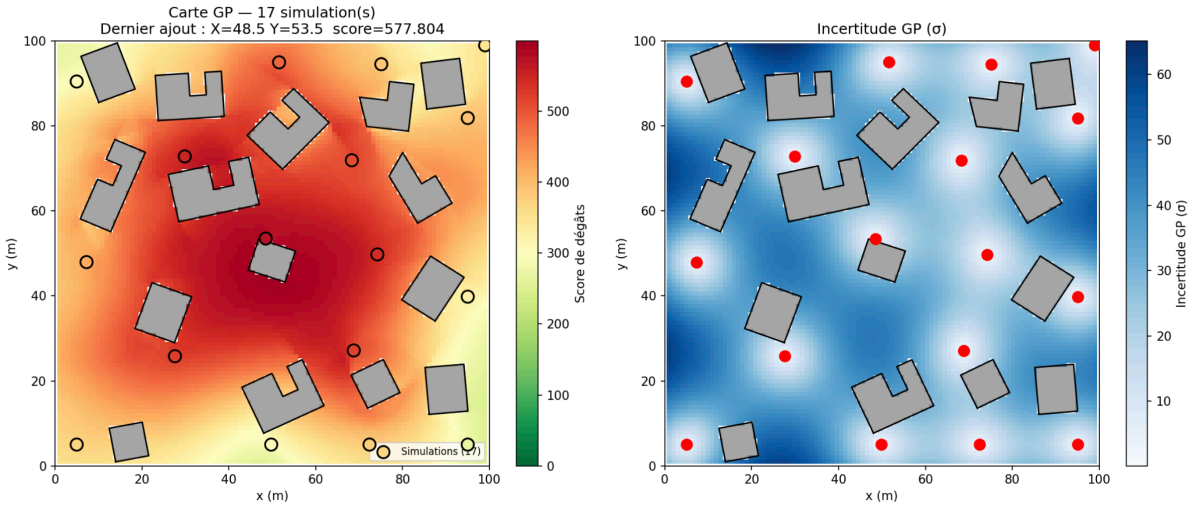


Figure 4: GP damage map (left) and posterior standard deviation (right) on the 2D test scene, after 17 of the 19 ARMEN simulations (circles and dots). The damage value at each position is the criticality-weighted score S of (8) for a detonation at that position; gray polygons are buildings. Uncertainty collapses around simulated points and stays high in unexplored areas. The labels come from the original French tool: *Score de dégâts* = damage score, *Incertitude* = uncertainty, *Dernier ajout* = last added point.

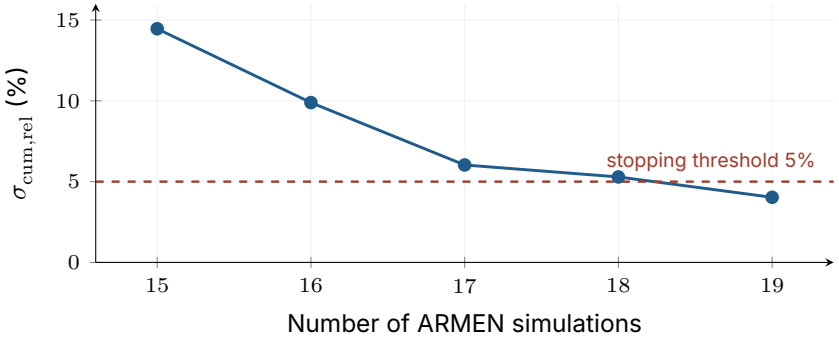


Figure 5: Relative cumulative uncertainty on the route corridor as simulations are added.

only the danger peak, at the cost of a potentially very long route. Delta-Stepping optimizes the integral of risk instead. Overall, **19 well-chosen simulations**, 0.2 % of the 10 000 positions of the evaluation grid, were enough for a stable route (Figure 4). This supported the move to 3D.

5.3 Status and performance in 3D

The 3D pipeline runs end to end on the partner's cluster, and six complete ARMEN simulations have been integrated into the GP. A quantitative convergence study in 3D, of the kind performed in 2D, was beyond the time available at delivery. Table 5 gives typical durations and Table 6 the complexity of each step.

5.4 Scalability of parallel Delta-Stepping

The path search is modest compared with the capacity of the supercomputer: a few hundred thousand nodes on a few dozen processes. The question is therefore not to fill the machine. It is to exploit its low-latency interconnect, and to anticipate the much larger urban domains where it would become necessary.

Step	2D	3D
Scene parsing	∼2 min	∼2 min
ARMEN simulation	∼10 min	**∼2 h**
Post-processing	∼3 min	∼8 min
KB surrogate (50×50, parallel)	∼2 min	∼2 min
GP update	<5 s	<5 s
Directional A* (50×50)	<10 s	<10 s

Table 5: Typical durations. The 3D simulation is the bottleneck.

Step	Complexity
KB, per node	$O(\lvert\mathcal{P}\rvert\,(N_{\text{bat}} + n_s))$ for masking and $O(\lvert\mathcal{P}\rvert \log \lvert\mathcal{P}\rvert)$ for smoothing, with $\lvert\mathcal{P}\rvert$ probes and $n_s = 20$ ray samples
GP fit	$O(n_{\text{obs}}^3)$ (Cholesky factorization)
GP prediction	$O(n_{\text{obs}} N_v)$ for the mean, $O(n_{\text{obs}}^2 N_v)$ for the variances
Directional A*	$O(NN_d \log(NN_d))$, with a constant branching factor of 8

Table 6: Algorithmic complexity. For $N = 2\,500$, $N_d = 8$ and $n_{\text{obs}} = 6$, all post-simulation steps are near-instantaneous.

Nature of the figures. The figures in this section are **estimates**, derived from the graph sizes and from a latency–bandwidth cost model; they are not measurements. A full benchmarking campaign did not fit in a three-month project and was not its objective. The aim here is to identify where the parallel path search would break down, and how it could be fixed.

Problem size. Table 7 gives graph dimensions. Going from 2D to 3D multiplies nodes by N_z and edges by about $3.25\,N_z$ (8 to 26 neighbors). The vehicle mode brings the effective size back to the 2D case.

Configuration	Nodes	Edges	Memory (COO)
2D 100×100	10 000	∼80 000	∼1.3 MB
3D $100 \times 100 \times 30$	300 000	∼8 M	∼130 MB
3D $200 \times 200 \times 50$	2 000 000	∼52 M	∼830 MB
3D $500 \times 500 \times 100$	25 000 000	∼650 M	∼10 GB
Vehicle 100×100 (surface)	10 000	∼80 000	∼1.3 MB

Table 7: Graph dimensions, assuming COO storage (int32 indices, float64 weights, 16 bytes per edge).

Strong scaling. The speed-up comes from processing the B nodes of a bucket concurrently, about B/P per process, so it is bounded by the mean bucket size. A large Δ fills buckets but adds redundant relaxations; a small Δ gives narrow buckets and less parallelism. Each iteration performs three collectives: an all-reduce on one integer for the active bucket, an all-to-all for cross-partition relaxations, and an all-reduce on the distance vector of size $|V|$. The last one dominates, with cost

$$O\Big(\frac{|V|}{B_{\text{net}}} + \alpha_{\text{lat}} \log P\Big), \tag{22}$$

where B_{net} is the effective bandwidth and α_{lat} the latency.

- On an InfiniBand-class network, for $|V| = 300\,000$ (about 2.4 MB) it costs about 200 µs per iteration. Over about 1 000 iterations this adds up to 0.2 s, which is no longer negligible beyond 16–32 processes.

- On gigabit Ethernet (∼50 µs latency, ∼120 MB/s), the same exchange would cost about 20 ms per iteration and cap useful parallelism near 4 processes.

Weak scaling. Local relaxation work stays constant at N/P nodes per process, but the global all-reduce grows linearly with N. Exchanging only boundary distances with point-to-point messages would reduce this to $O(N^{2/3})$ in 3D, the surface of the subdomains.

Memory. Each process currently holds the whole graph (Table 8). On typical nodes with about 256 GB of RAM, replication remains viable up to about 10 GB per process, i.e., 25 million nodes. Beyond that, distributed storage with ghost layers is needed.

Grid	Per process (replicated)	Per process (distributed)
$100 \times 100 \times 30$	130 MB	∼33 MB ($P = 4$)
$200 \times 200 \times 50$	830 MB	∼52 MB ($P = 16$)
$500 \times 500 \times 100$	∼10 GB	∼160 MB ($P = 64$)

Table 8: Estimated memory per process, replicated versus distributed graph.

Partitioning and language. Slabs along z are simple but suboptimal on two counts. Their interfaces are full $N_x \times N_y$ planes regardless of P, and buildings concentrated in some layers unbalance the load. A graph partitioner such as METIS, combined with topology-aware placement on fat-tree networks, would reduce communication. The pure-Python implementation adds its own overheads:

- adjacency is stored as dictionaries of lists rather than compressed sparse rows;
- relaxation loops run about 100× slower than in C;
- messages are serialized as lists of tuples rather than contiguous buffers;
- neither SIMD vectorization nor the cache hierarchy is exploited.

Toward the optimal version — distributed path search

Current limit	Impact	Improvement
Global all-reduce of distances	Strong scaling capped at ∼16–32 processes	Point-to-point exchange of boundary nodes
Replicated graph	Viable up to ∼10 GB per process	Distributed storage with ghost layers
1D slabs along z	Imbalance, interface $N_x \times N_y$	METIS + topology-aware placement
Empirical Δ	Suboptimal for heterogeneous weights	Adaptive calibration
Pure Python	$\sim 100\times$ slower than C, no SIMD	CSR storage, Cython or C++ with native MPI
CPU only	GPU nodes unused	CUDA/OpenCL port of Delta-Stepping

6 Conclusion and perspectives

6.1 Contributions

The project delivered a complete route-optimization pipeline that couples high-performance hydrodynamic simulation with statistical learning, on a problem that is both computationally demanding and physically rich.

- **A modular architecture** chains scene generation, simulation, ICPE scoring, physics-based surrogate, Bayesian correction and path search. It supports partial restarts and module replacement, and it cleanly separates the slow component (ARMEN) from the fast ones (KB, GP).
- **Residual learning** of the gap between ARMEN and an analytical law converges to a reliable damage map with 5 to 20 simulations, where an exhaustive grid would need thousands and take months. The acquisition, coupled with the current route in 2D, spends simulations where uncertainty actually harms the decision.
- **A rigorous objective** combines cumulative damage and angular regularization, is physically motivated and nondimensionalized. Damage scoring is anchored on regulatory thresholds and differentiated criticality, which is closer to real damage mechanisms than uniform logarithmic laws.
- **Scalable path search**: an MPI Delta-Stepping runs on the partner's supercomputer, and a cost model indicates that a Python approach is viable at the scale of the 3D domains considered while documenting the limits to overcome. A provably optimal direction-aware A* handles terrain-following routes with turn and slope penalties.

6.2 Limitations and perspectives

Surrogate. The empirical fit used for the free-field law is valid only over a limited range of scaled distances (Section 3.4) and should be replaced by the standard KB formulation. Beyond this, KB ignores Mach reflection, street-canyon channeling and multiple reflections. Where residuals are large, the GP correction becomes insufficient, and the number of simulations needed may exceed the budget. A first step is to calibrate the masking and drag terms against ARMEN.

Toward neural operators. Neural operators, in particular Fourier Neural Operators and graph neural networks, are the most promising way to replace or complement the KB+GP surrogate. Physics-informed neural networks solve one instance at a time by minimizing physical residuals, and are ill-suited to 3D compressible Euler flows with shocks. Neural operators instead learn a *family* of solutions: a scene and a source position in, a 3D pressure field out, without retraining. That is exactly what an iterative optimization loop requires. Architectures such as JEPA or PINO remain immature for compressible-flow PDEs. Training would require roughly 2 000 to 10 000 ARMEN runs, a substantial but achievable investment over the life cycle of such a tool.

More data in 3D. Bringing the number of 3D observations to 15–20 would refine the GP map and reduce σ_{post} in the zones that matter, and would allow the convergence study performed in 2D.

Kernel geometry. The Euclidean distance of stationary kernels underestimates acoustic isolation behind obstacles. Physically informed input features, non-stationary or deep kernels are the PSD-safe ways to inject geometry (Section 3.5).

Usability and constraints. The remaining steps are to finalize the interactive interface for parameter input and route display, to test larger scenes with more buildings, and to add practicability constraints such as maximum slope and forbidden zones.

Real-world bridge. The pipeline could run on any city with an autonomous chain from GPS coordinates to a route:

- geometric acquisition from mapping services;
- automatic ICPE categorization of buildings;
- 3D mesh reconstruction by footprint extrusion and Delaunay triangulation;
- live data on topography and human occupancy.

This bridge between the virtual case study and real urban environments would be the natural continuation of the project.

References

[1] ioMosaic, *Risk-Based Approach – Damage Criteria*.

[2] W. P. M. Mercx, *Damage Criteria and Safety Distances for Industrial Explosions*, IChemE, 2010.

[3] M. Larcher et al., Design of blast-loaded glazing windows and facades, *Engineering Failure Analysis*, 2016.

[4] Y. Huang et al., Blast risk assessment of wood residential buildings, *Journal of Structural Engineering*, 2020.

[5] NIST, *Community Resilience Planning Guide for Buildings and Infrastructure Systems*, NIST SP 1190, 2016.

[6] NIST, *Resilience for Critical Facilities*, NIST Technical Note, 2023.

[7] IDMC, *Global Report on Internal Displacement 2025*, 2025.

[8] UNICEF, *Learning Interrupted: Climate-Related School Disruptions in 2024*, 2025.

[9] FEMA, *Design Guide for Improving Critical Facility Safety from Flooding and High Winds*, FEMA 543, 2007.

[10] OECD, *Good Governance for Critical Infrastructure Resilience*, OECD Publishing.

[11] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.

[12] A. I. J. Forrester, A. Sóbester and A. J. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide*, Wiley, 2008.

[13] C. N. Kingery and G. Bulmash, *Airblast Parameters from TNT Spherical Air Burst and Hemispherical Surface Burst*, Technical Report ARBRL-TR-02555, U.S. Army Ballistic Research Laboratory, 1984.

[14] U.S. Department of Defense, *Structures to Resist the Effects of Accidental Explosions*, Unified Facilities Criteria UFC 3-340-02, 2008.

[15] M. M. Swisdak, Simplified Kingery airblast calculations, *Proc. 26th DoD Explosives Safety Seminar*, 1994.

[16] M. L. Stein, *Interpolation of Spatial Data: Some Theory for Kriging*, Springer, 1999.

[17] J. Mockus, *Bayesian Approach to Global Optimization*, Kluwer, 1989.

[18] N. Srinivas, A. Krause, S. M. Kakade and M. Seeger, Gaussian process optimization in the bandit setting: no regret and experimental design, *Proc. ICML*, 2010, pp. 1015–1022.

[19] J. A. Sethian, *Level Set Methods and Fast Marching Methods*, 2nd ed., Cambridge University Press, 1999.

[20] E. W. Dijkstra, A note on two problems in connexion with graphs, *Numerische Mathematik* 1, 269–271, 1959.

[21] P. E. Hart, N. J. Nilsson and B. Raphael, A formal basis for the heuristic determination of minimum cost paths, *IEEE Trans. Systems Science and Cybernetics* 4(2), 100–107, 1968.

[22] U. Meyer and P. Sanders, Δ-stepping: a parallelizable shortest path algorithm, *Journal of Algorithms* 49(1), 114–152, 2003.